Practical 11

Aim : Study and apply real life application of Blockchain.

Introduction

Blockchain is a distributed digital ledger that records transactions in a secure and transparent manner. Instead of storing transaction records on a single centralized server, copies of the ledger are maintained by multiple participants in the network.

A transaction is grouped with other transactions into a **block**. After verification, the block is added to the blockchain. Once recorded, the transaction is extremely difficult to alter.

Real-Life Application: Cryptocurrency Transaction

A common real-life application of blockchain is **cryptocurrency transactions**, such as transferring cryptocurrency from one user to another.

For example:

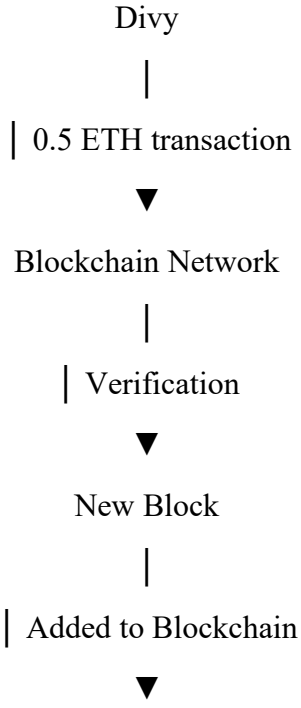
User A → User B: 0.5 ETH

The transaction follows these steps:

1. **Transaction Creation**
   User A enters User B's wallet address and the amount to transfer.
2. **Digital Signature**
   User A digitally signs the transaction using their private key.
3. **Broadcasting**
   The transaction is broadcast to the blockchain network.
4. **Transaction Verification**
   Network participants verify the transaction and check whether User A has sufficient funds.
5. **Block Formation**
   The verified transaction is included in a new block.
6. **Consensus**
   The blockchain network reaches agreement on the validity and ordering of the block.
7. **Block Addition**
   The block is added to the blockchain.
8. **Transaction Completion**
   The balance of User A decreases and User B's balance increases.

Example

Suppose Divy wants to send **0.5 ETH** to another person's wallet.



Recipient's Wallet

The transaction can then be viewed on the blockchain using the transaction hash.

Simple implementation using Python

```
import datetime

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = datetime.datetime.now()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        text = str(self.index) + str(self.timestamp) + self.data + self.previous_hash

        hash_value = 0
        for ch in text:
            hash_value = (hash_value * 31 + ord(ch)) % 1000000007

        return str(hash_value)

block1 = Block(1, "BlockChain", "0")

block2 = Block(2, "Bitcoin", block1.hash)
```

```
print("Block 1")
print("Index        :", block1.index)
print("Timestamp    :", block1.timestamp)
print("Data         :", block1.data)
print("Previous Hash :", block1.previous_hash)
print("Hash         :", block1.hash)

print("\nBlock 2")
print("Index        :", block2.index)
print("Timestamp    :", block2.timestamp)
print("Data         :", block2.data)
print("Previous Hash :", block2.previous_hash)
print("Hash         :", block2.hash)

print("\nConnection:")
print("Block 2 Previous Hash == Block 1 Hash")
print(block2.previous_hash == block1.hash)
```

```
Block 1
Index         : 1
Timestamp     : 2026-07-17 06:22:45.650677
Data          : BlockChain
Previous Hash : 0
Hash          : 312629173

Block 2
Index         : 2
Timestamp     : 2026-07-17 06:22:45.650758
Data          : Bitcoin
Previous Hash : 312629173
Hash          : 640155660

Connection:
Block 2 Previous Hash == Block 1 Hash
True
```

Advantages

- **Transparency:** Transactions can be publicly verified on public blockchains.
- **Security:** Cryptographic techniques protect transactions.
- **Immutability:** Confirmed records are difficult to modify.
- **Decentralization:** No single organization needs to maintain the entire ledger.
- **Traceability:** Transactions can be tracked using their blockchain records.
- **Reduced intermediaries:** Direct blockchain transactions can reduce dependence on traditional intermediaries.

Other Real-Life Applications

Blockchain-based transactions are also used or explored in:

Application	Use of Blockchain
Banking	Cross-border payments
Supply Chain	Tracking movement and ownership of goods
Real Estate	Recording property transactions
Healthcare	Secure sharing of medical records
Voting	Recording and verifying votes
Insurance	Automated claim processing
Digital Identity	Verification of identity
E-commerce	Secure digital payments

Theory (brief)

A private Ethereum network is a blockchain instance run independently of the public Ethereum mainnet or testnets, typically for local development and testing. It behaves like a real Ethereum network — supporting accounts, transactions, and smart contracts — but is fully controlled by whoever runs it, with no connection to outside miners or validators.

Smart contracts are self-executing programs deployed on the blockchain. Once deployed, their functions can be called externally to read or modify on-chain state, and the contract enforces its own rules on every call — including restricting certain actions to specific accounts.

Key concepts:

- **Geth (Go-Ethereum)** — an Ethereum client written in Go; running it in --dev mode spins up a private, single-node blockchain instantly, complete with a pre-funded developer account
- **RPC endpoint** — the HTTP interface (http://127.0.0.1:8545 by default) that Geth exposes, allowing external tools like Remix to send transactions and queries to the node
- **Geth JavaScript console** — an interactive console (geth attach) used to inspect the running node directly — checking network version, block number, and available accounts
- **Owner-based access control** — a common smart contract security pattern where the address that deployed the contract is stored as owner, and sensitive functions check msg.sender == owner before executing
- **require() statement** — a Solidity guard clause that reverts the transaction (undoing all state changes and refunding unused gas) if its condition is false, along with an optional error message

Tools used for this practical

- Go-Ethereum (Geth) — to create and run the private Ethereum network

- Command Prompt — to start Geth and attach the console
- Remix IDE (remix.ethereum.org) — to write, compile, and deploy the smart contract
- Web browser — to access Remix and interact with the deployed contract

Implementation

- Confirm Geth is installed correctly by running geth version in Command Prompt.

```
PS C:\Users\priya> geth version
Geth
Version: 1.17.6-stable
Git Commit: 3d84c6b2e3e9be032f8f2dcd9e900f89391fcdab
Git Commit Date: 20260923
Architecture: amd64
Go Version: go1.27.1
Operating System: windows
GOPATH=C:\Users\priya\go
GOROOT=
```

- Create a dedicated folder to hold the private network's data: mkdir C:\PrivateEthereum then cd C:\PrivateEthereum.

```
PS C:\Users\priya> mkdir PrivateEthereum


    Directory: C:\Users\priya


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        25-09-2026     07:40                PrivateEthereum


PS C:\Users\priya> cd .\PrivateEthereum\
```

- Start the private network in developer mode: geth --datadir C:\PrivateEthereum --dev --http --http.api eth,web3,net --http.corsdomain "https://remix.ethereum.org". This launches a single-node Ethereum instance with a pre-funded developer account and keeps an HTTP RPC endpoint open for external tools to connect to.

```
PS C:\Users\priya\PrivateEthereum> geth --datadir C:\PrivateEthereum --dev --http --http.api eth,web3,net --http.corsdomain "https://remix.ethereum.org"
INFO [09-25|07:41:26.804] Starting Geth on Ethereum mainnet...
INFO [09-25|07:41:26.823] Maximum peer count                       ETH=50 total=50
INFO [09-25|07:41:26.834] Set global gas cap                       cap=50,000,000
INFO [09-25|07:41:26.834] Engine API maximum reorg depth           depth=32
INFO [09-25|07:41:26.972] Using developer account                  address=0x71562b71999873DB5b286dF957af199Ec94617F7
INFO [09-25|07:41:26.973] Defaulting to pebble as the backing database
INFO [09-25|07:41:26.976] Allocated cache and file handles         database=C:\PrivateEthereum\geth\chaindata cache=2.00GiB handles=8192 version=v2
INFO [09-25|07:41:27.073] Opened ancient database                  database=C:\PrivateEthereum\geth\chaindata\ancient\chain readonly=false
INFO [09-25|07:41:27.073] Opened Era store                         datadir=C:\PrivateEthereum\geth\chaindata\ancient\chain\era
WARN [09-25|07:41:27.076] Head block is not reachable
INFO [09-25|07:41:27.076] Freezer shutting down
INFO [09-25|07:41:27.115] Initializing the KZG library             backend=gokzg
INFO [09-25|07:41:27.117] Allocated trie memory caches             clean=614.00MiB dirty=1.00GiB
INFO [09-25|07:41:27.118] Using pebble as the backing database
INFO [09-25|07:41:27.118] Allocated cache and file handles         database=C:\PrivateEthereum\geth\chaindata cache=2.00GiB handles=8192 version=v2
INFO [09-25|07:41:27.168] Opened ancient database                  database=C:\PrivateEthereum\geth\chaindata\ancient\chain readonly=false
INFO [09-25|07:41:27.168] Opened Era store                         datadir=C:\PrivateEthereum\geth\chaindata\ancient\chain\era
WARN [09-25|07:41:27.168] Head block is not reachable
INFO [09-25|07:41:27.169] State schema set to default              scheme=path
INFO [09-25|07:41:27.175] Initialising Ethereum protocol           network=1337 dbversion=<nil>
WARN [09-25|07:41:27.175] Sanitizing invalid node buffer size      provided=1.00GiB updated=256.00MiB
```

```
WARN [09-25|07:41:27.569]       your computer or losing power will wipe your entire block data and chain state for
WARN [09-25|07:41:27.569]       your dev environment.
WARN [09-25|07:41:27.569]    3. A random, pre-allocated developer account will be available and unlocked as
WARN [09-25|07:41:27.569]       eth.coinbase, which can be used for testing. The random dev account is temporary,
WARN [09-25|07:41:27.569]       stored on a ramdisk, and will be lost if your machine is restarted.
WARN [09-25|07:41:27.569]    4. Mining is enabled by default. However, the client will only seal blocks if transactions
WARN [09-25|07:41:27.570]       are pending in the mempool. The miner's minimum accepted gas price is 1.
WARN [09-25|07:41:27.570]    5. Networking is disabled; there is no listen-address, the maximum number of peers is set
WARN [09-25|07:41:27.570]       to 0, and discovery is disabled.
WARN [09-25|07:41:27.570]
INFO [09-25|07:41:27.570] Starting peer-to-peer node               instance=Geth/v1.17.6-stable-3d84c6b2/windows-amd64/go1.27.1
WARN [09-25|07:41:27.570] P2P server will be useless, neither dialing nor listening
INFO [09-25|07:41:27.595] IPC endpoint opened                      url=\\.\pipe\geth.ipc
INFO [09-25|07:41:27.595] New local node record                    seq=1,790,302,287,594 id=0d5ac26dde7620ae ip=127.0.0.1 udp=0 tcp=0
INFO [09-25|07:41:27.595] Started P2P networking                   self=enode://1bf661b44b0c876f891845814738bb8494bc6da23cbffea8e0a14b02d8c11b09576c2703116b
bcaccaf6a970f7bcb3437a19158b4ebe1ebaa80458c14d71ed27@127.0.0.1:0
INFO [09-25|07:41:27.599] HTTP server started                      endpoint=127.0.0.1:8545 auth=false prefix= cors=https://remix.ethereum.org vhosts=localho
st
INFO [09-25|07:41:27.601] Started log indexer
```

- In a separate Command Prompt window, attach to the running node's console: geth attach http://127.0.0.1:8545.

```
PS C:\Users\priya\PrivateEthereum> geth attach http://127.0.0.1:8545
Welcome to the Geth JavaScript console!

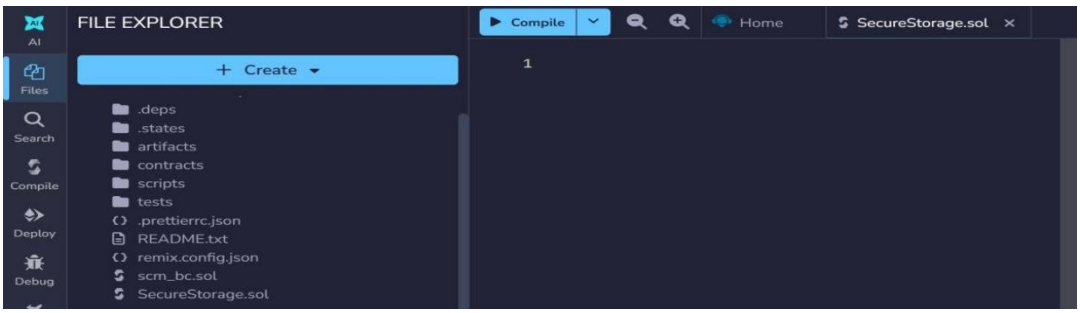
instance: Geth/v1.17.6-stable-3d84c6b2/windows-amd64/go1.27.1
at block: 0 (Thu Jan 01 1970 05:30:00 GMT+0530 (IST))
 modules: eth:1.0 net:1.0 rpc:1.0 web3:1.0

To exit, press ctrl-d or type exit
>
```

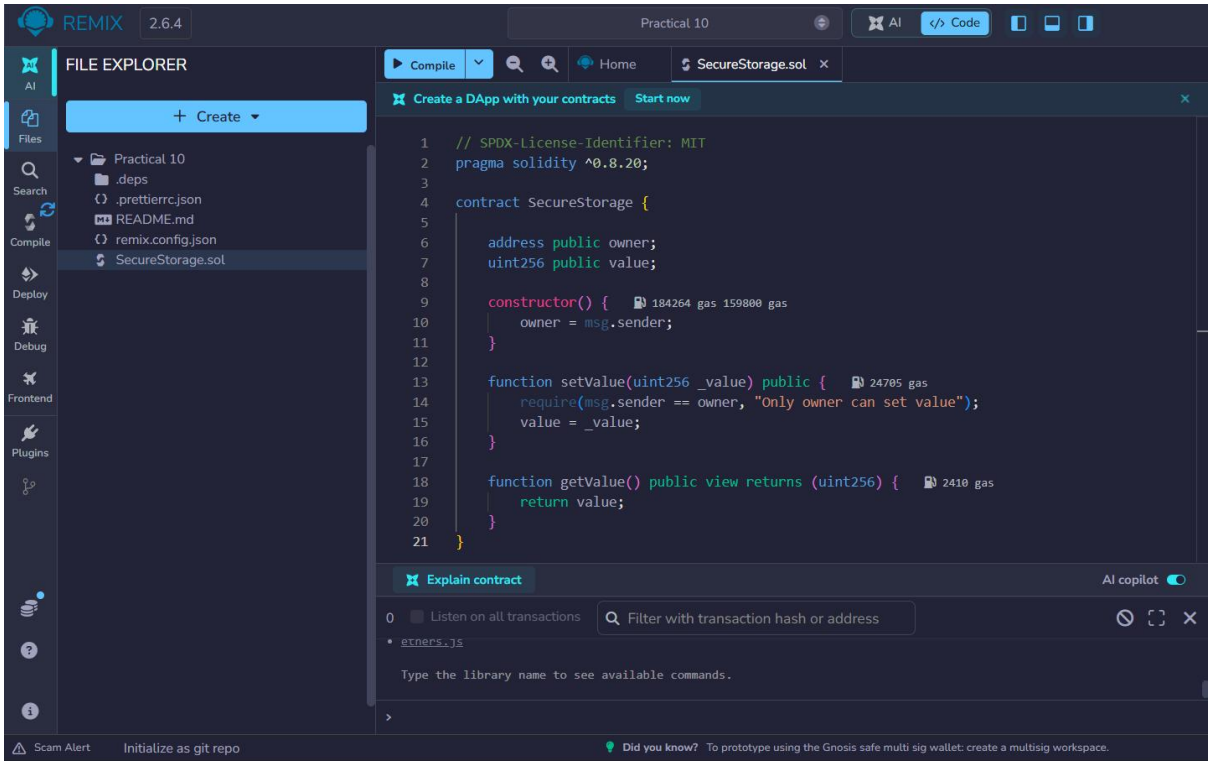
- From the console, verify the network is live and check its state: net.version (confirms the private chain ID), eth.blockNumber (current block height), and eth.accounts (lists the developer account created automatically).

```
To exit, press ctrl-d or type exit
> net.version
"1337"
> eth.blockNumber
0
> eth.accounts
["0x71562b71999873db5b286df957af199ec94617f7"]
>
```

- Open Remix IDE in a browser and create a new file, SecureStorage.sol.



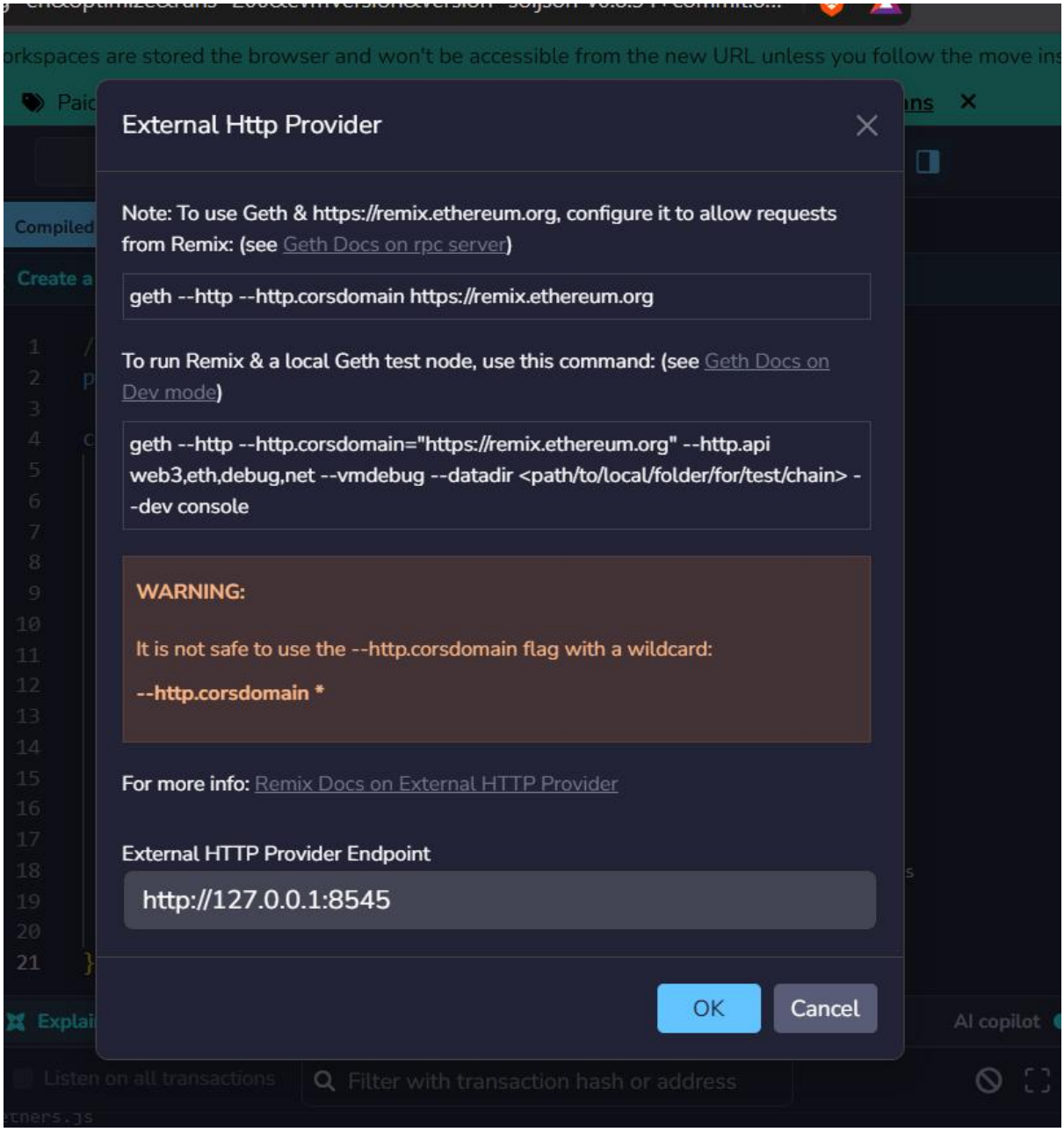
- Write the contract: it declares a public owner address set in the constructor to whoever deploys it, a public value variable, a setValue() function restricted to the owner via a require() check, and a getValue() view function open to anyone.



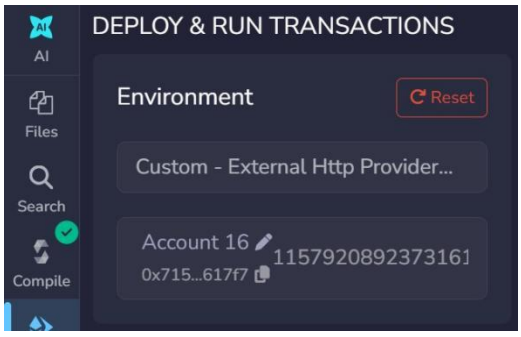
- Open the Solidity Compiler tab, select a compiler version matching pragma solidity ^0.8.20, and compile the contract.



- Go to Deploy & Run Transactions, and under Environment select "Custom - External Http Provider", pointing it to http://127.0.0.1:8545 so Remix connects to the private network instead of its default in-browser VM. The developer account from the private chain appears as the active account.



- Click Deploy; the contract is mined onto the private chain and appears under Deployed Contracts with its on-chain address.



- Call setValue with a test number, then call getValue to confirm the stored value is correctly written to and read from the blockchain.



